Applications in the Real World: Performance Analysis of Cuckoo Hashing and its Extensions

Pratyush Jain, Aditya Raghuram, Aaradhy Sharma, Arnav Neekhra, and Jasmehar Chadha

Abstract—Hash tables are foundational to modern computing, but traditional designs degrade at high load due to collision clustering. Cuckoo hashing guarantees worst-case constant-time lookup, yet its canonical two-table form is limited to a load factor of approximately 50%. A family of extensions—bucketization, stash augmentation, d-ary generalization, and cache-optimized variants—has been proposed to overcome this wall while preserving the worst-case guarantee. However, empirical comparisons across this family, together with alternative open-addressing schemes, remain fragmented in the literature, and most published evaluations rely solely on synthetic uniform-random keys. We present a systematic empirical evaluation of ten hash table variants—nine dynamic (standard, bucketized, stashed, and d-ary cuckoo; separate chaining; linear, quadratic, Robin Hood, and hopscotch probing) plus Fredman-Komlós-Szemerédi (FKS) perfect hashing as a static baseline—paired with four hash families (MurmurHash3, xxHash32, FNV-1a, and Carter-Wegman universal). Benchmarks use both synthetic distributions and real Wikipedia pageview data. We formulate five falsifiable hypotheses (H1–H5) covering load factor ceilings, stash effectiveness, synergy, practical competitiveness, and distribution sensitivity, and test them using 126 unit tests and $340+$ benchmark configurations executed via the Java Microbenchmark Harness (JMH), yielding over $9{,}500$ measured samples. Our results confirm H1 (bucketized cuckoo with $B = 4$ achieves $\sim 98\%$ load factor, matching Erlingsson et al.), H2–H3 (the stash behaves as predicted by theory), and H5 (cuckoo schemes are 11–34% slower on real string keys than on synthetic integer keys), but *falsify* H4: bucketized cuckoo is approximately 60% slower than linear probing for positive lookups at equal data size. This falsification reveals a precise trade-off: bucketization is a memory-efficiency optimization, not a speed optimization. We also report a scale-dependent hash function sensitivity for FNV-1a that is negligible at $100{,}000$ entries but drops its lookup throughput to $\sim 60\%$ of MurmurHash3, xxHash32, and Carter-Wegman universal at $500{,}000$ entries. Standard cuckoo's tombstone-free $O(1)$ deletion is empirically validated as competitive with separate chaining and superior to all probing baselines.

Index Terms—cuckoo hashing, bucketization, stash, hash functions, JMH, load factor, real-world benchmarking

1 Introduction

1.1 Background and Motivation

Hash tables underpin virtually every database, key–value store, compiler, network router, and language runtime. Their theoretical appeal—expected $O(1)$ insertion, lookup, and deletion—makes them ubiquitous in latency-sensitive systems, from CPU branch-prediction tables to distributed in-memory caches such as Memcached and Redis. However, traditional collision-resolution strategies such as separate chaining and linear probing suffer *worst-case* degradation to $O(n)$ when collisions cluster: a single long probe sequence or an unexpectedly long chain can turn a routine lookup into a cache-unfriendly traversal of many hundreds of bytes. In workloads where tail latency matters—for example, interactive web services, financial trading systems, or high-frequency packet processing—this variability is unacceptable.

Cuckoo hashing, introduced by Pagh and Rodler [1], offers an elegant alternative: each key resides in one of exactly two candidate slots, giving a *worst-case* $O(1)$ lookup guarantee of at most two memory accesses regardless of table occupancy or input ordering. In the decades since, cuckoo hashing has seen substantial industrial adoption: Intel's data-plane development kit uses it for network flow tables, major key–value stores have built on its principles [2], [3], and its descendants power cuckoo filters and learned index structures.

This theoretical appeal, however, is paid for in space. Standard two-table cuckoo hashing fails to accept new keys once occupancy exceeds approximately 50%, a phenomenon known as the "load factor wall." Beyond this threshold, insertion failure probability transitions sharply from vanishing to near-certain—an artifact of the underlying cuckoo graph's phase transition. A rich body of subsequent work has attempted to break this wall while preserving the worst-case guarantee, producing a family of extensions including *bucketization* (placing multiple items per position) [2], [4], the *stash* (a small auxiliary array for overflow) [5], *d-ary* generalization (using more than two hash functions) [6], [7], and cache-optimized variants [3], [8].

1.2 Research Gap

Despite the theoretical maturity of this area, empirical comparisons across the full family of cuckoo-family schemes—*together with alternative open-addressing strategies such as hopscotch [9] and Robin Hood [10]*—remain fragmented in the literature. Three specific gaps motivate this work.

First, many published evaluations compare only within the cuckoo family itself, leaving unanswered the question of how bucketized or stashed cuckoo fare against genuinely competitive single-hash alternatives like hopscotch or Robin Hood. Second, most evaluations use synthetic uniform-random keys exclusively, which—as we demonstrate—can

• *The authors are with the School of Engineering, Shiv Nadar Institution of Eminence, Delhi-NCR, India.*
E-mail: {pj825, ar000, as000, an000, jc000}@snu.edu.in

mask distribution-sensitivity effects that are material in deployed systems. Third, the role of the underlying hash function in cuckoo performance is often dismissed as a constant-factor concern; our results reveal a scale-dependent regime in which weaker-avalanche functions produce markedly different outcomes.

1.3 Contributions

This paper makes three contributions:

- *Breadth.* We implement and benchmark nine dynamic hash table variants (four cuckoo-family and five open-addressing/chaining baselines) plus FKS perfect hashing as a static comparison point, behind a unified Java interface. Four pluggable hash families (MurmurHash3, xxHash32, FNV-1a, Carter-Wegman universal) allow us to separate scheme-level from hash-function-level effects. This produces a head-to-head comparison at a scale not, to our knowledge, previously published in a single study.
- *Real-world data.* We benchmark every variant on a real Wikipedia pageview dataset of variable-length string keys, in addition to four synthetic distributions (uniform, sequential, Zipfian, and hotspot). This directly addresses published concerns that synthetic benchmarks overstate hash table robustness.
- *Falsifiable hypotheses.* We formulate five explicit hypotheses (H1–H5) with falsification criteria tied to specific measurable thresholds, test each empirically, and report the falsification of H4 as a central finding: *bucketization is a memory-efficiency optimization, not a speed optimization.* This is, we believe, a useful corrective to the narrative common in the cuckoo-hashing literature that positions bucketization as a universal improvement.

In addition, we report a number of secondary findings of independent interest: (a) standard cuckoo's tombstone-free $O(1)$ deletion is empirically competitive with separate chaining and strictly better than all probing baselines; (b) hopscotch hashing is a consistently strong all-rounder that has received less attention than its performance merits; (c) a scale-dependent hash function sensitivity appears only at tables large enough to exceed L2 cache, offering an empirical caveat to the folklore claim that hash quality is irrelevant for cuckoo on integer keys; and (d) a methodological correction to our lookup benchmark, which initially conflated "target load factor" with "number of inserted elements" across variants with different load-factor ceilings.

1.4 Paper Organization

Section 2 surveys the relevant literature on cuckoo hashing and its alternatives. Section 3 describes each of the ten implemented hash table variants together with the four hash functions. Section 4 states the five hypotheses, enumerates the benchmark suite, describes the real-data acquisition, and documents a methodological correction. Section 5 presents results organized hypothesis-by-hypothesis with pgfplots figures and tables. Section 6 concludes with design guidance, limitations, and directions for future work.

2 Literature Survey

2.1 Foundations of Cuckoo Hashing

Pagh and Rodler [1] introduced cuckoo hashing as a technique using two hash tables T_1 and T_2 and two independent hash functions h_1, h_2. Every key x lives in either $T_1[h_1(x)]$ or $T_2[h_2(x)]$. Insertions may trigger a chain of displacements—new keys "evict" existing ones, which attempt to settle in their alternate position. The authors proved that for load factor $\alpha < 1/2$, insertion succeeds in expected $O(1)$ time with failure probability $O(1/n^2)$ per insertion, and that the expected cost of a forced rebuild is bounded. Crucially, lookup requires *at most two memory accesses*, giving a deterministic worst-case $O(1)$ guarantee unmatched by open-addressing competitors.

2.2 Bucketization

To overcome the 50% load factor wall, Erlingsson, Manasse, and McSherry [4] proposed *bucketized* (set-associative) cuckoo hashing, in which each of the two table positions is a *bucket* holding B items. Intuitively, a key is unable to be placed only if both of its buckets are entirely full, which occurs with dramatically lower probability than a single-slot conflict. Empirical and theoretical analyses show load factor thresholds of $\sim 86\%$ for $B = 2$, $\sim 95\%$ for $B = 4$, and $\sim 98\%$ for $B = 8$. Fan et al. showed in *MemC3* [2] that a 4-way bucketized design can deliver a $3\times$ throughput improvement and a 30% memory-footprint reduction over stock Memcached. Li et al. [3] extended this to a concurrent variant (*libcuckoo*) with fine-grained lock striping and a breadth-first search strategy for finding displacement paths.

2.3 Stash Extension

Kirsch, Mitzenmacher, and Wieder [5] proposed augmenting cuckoo hashing with a constant-size auxiliary array—a *stash*—to catch items that cannot be placed after a bounded number of displacement attempts. They proved that with a stash of size s, the insertion failure probability drops from $\Theta(1/n)$ to $O(1/n^{s+1})$. Intuitively, a stash of only three or four entries reduces the expected rehash frequency to a level where it becomes an engineering non-issue for tables of practical size. The cost is a linear scan of the stash on lookup, but this is applied only when both table positions miss, and it touches a structure small enough to reside in L1 cache.

2.4 d-ary Cuckoo Hashing

Fotakis, Pagh, Sanders, and Spirakis [6] generalized cuckoo hashing to use d hash functions (and hence d candidate positions) per key. As d grows, the load factor threshold c_d^* rises rapidly: $c_2^* \approx 0.5$, $c_3^* \approx 0.918$, $c_4^* \approx 0.977$. A long-standing open question—whether d-ary insertion maintains $O(1)$ expected time *up to* the full threshold, not just the lower peeling threshold—was recently resolved affirmatively by Bell and Frieze [7]. In practice, d-ary is compared with bucketized cuckoo as two different ways to reach high load factors, with the trade-off that d-ary incurs d independent cache misses per lookup while bucketization incurs only one or two (plus intra-bucket scans).

2.5 Alternative Open-Addressing Schemes

Hopscotch hashing [9] maintains a per-bucket bitmap of an H-slot neighborhood, guaranteeing that every key lies within a bounded distance of its home bucket. Lookups scan only the set bits of the bitmap, giving a worst-case $O(H)$ bound with excellent cache locality. *Robin Hood* hashing [10] is a linear-probing variant that equalizes probe distances by having each new key "steal" a slot from an item with a shorter probe distance, reducing the maximum probe length to $O(\log n)$ with high probability. Both schemes avoid tombstones via backward-shift deletion.

2.6 Hash Functions and Provable Families

Cuckoo hashing is sensitive to hash function *quality* (avalanche behavior): correlated h_1, h_2 outputs produce non-uniform bucket loads, lengthening displacement chains. MurmurHash3 is the de facto academic default, passing all SMHasher [11] tests and widely used in cuckoo-family papers. xxHash [11], while nominally faster in SIMD-enabled C/C++, offers no meaningful speedup on the JVM. FNV-1a is simpler but fails some SMHasher avalanche tests; whether this matters for cuckoo hashing in practice is an open empirical question that we address. Orthogonal to these engineered mixers, *universal hashing* [12] provides theoretical collision bounds: the Carter-Wegman family $h_{a,b}(x) = ((ax + b) \bmod p)$ is provably 2-universal for random (a, b), and the Fredman-Komlós-Szemerédi (FKS) construction [13] builds on universal hashing to produce a static *perfect* hash with zero collisions. We include both families in our evaluation to contrast "theoretically sound" with "empirically tuned."

2.7 Benchmarking Standards and Real Data

The Yahoo Cloud Serving Benchmark (YCSB) [14] defines six canonical workloads (A–F) spanning read-heavy, write-heavy, and scan patterns. For hash tables, workloads A, B, C, and F are most relevant since they consist of point operations. YCSB provides seven key-selection distributions: uniform, Zipfian (default skew $\theta = 0.99$), scrambled Zipfian, latest, hotspot, exponential, and sequential. YCSB's operation-mix framework has been adopted as an industry norm for key–value store evaluation. However, YCSB remains a *synthetic* workload: keys are generated from parametric distributions at run time rather than replayed from real traces. Real access patterns in production systems exhibit multi-modal distributions, temporal non-stationarity (trending topics shift hot keys over time), and structural correlations (similar keys such as sequential IDs or related IP prefixes may hash to correlated slots). Pure Zipfian distributions do not capture these effects.

Freely available real-world datasets used in recent hash-table literature include Wikipedia pageview logs [15], anonymized CAIDA backbone-link traces (requiring institutional access), the WorldCup 1998 HTTP log, and the TPC-H dbgen output. Leading systems venues—OSDI, NSDI, VLDB, EuroSys—increasingly expect hash table papers to include at least one real-world dataset alongside synthetic benchmarks. Our evaluation addresses this convention by testing every implemented variant on a 38 MB Wikipedia pageview dump, extracting the first 100,000 unique English-language page titles as string keys.

2.8 Scale-Dependent Effects and Benchmarking Methodology

A parallel strand of literature examines benchmarking methodology itself. Rigorous hash-table benchmarking on the JVM requires awareness of dead-code elimination, constant folding, and loop-hoisting optimizations that can invalidate naive timing code. The Java Microbenchmark Harness (JMH) [16] was developed by the OpenJDK team to address these concerns, providing infrastructure for warmup iterations, multiple forks (each in a fresh JVM to eliminate profile pollution), and automatic blackhole consumption of benchmark return values. Systems-level papers additionally measure hardware performance counters via `perf` or Intel VTune, but these tools are unavailable on Apple Silicon, restricting our analysis to throughput and allocation metrics.

A recurring observation in the benchmarking literature is that relative performance between schemes can invert at different table sizes—a small table that fits in L1 cache may favor cache-friendly designs that lose their advantage at L3-exceeding scales. We explicitly vary n across three orders of magnitude to expose these effects.

3 Techniques

We implemented ten hash table variants behind a unified Java interface `CuckooHashTable<K,V>` with six methods: `get`, `put`, `remove`, `size`, `loadFactor`, and `getStats`. Nine variants are dynamic; the tenth (FKS perfect hashing) is static and throws `UnsupportedOperationException` on `put/remove`. We describe each technique below, grouped by family.

3.1 Standard Two-Table Cuckoo Hashing

Our implementation maintains two parallel arrays `table1`, `table2` of capacity $\lceil 2.2 \cdot n \rceil$ each, where n is the expected size. On insertion, we probe $h_1(x)$ in `table1`; if occupied, we evict the incumbent and attempt to place it at its $h_2(\cdot)$ position in `table2`, alternating tables until an empty slot is found or a *maxLoop* limit of $6 \log_2 n$ displacements is exceeded. On maxLoop exceedance, we regenerate the two hash-function seeds and rebuild; after ten unsuccessful seed regenerations, we double capacity. Deletion is trivial: clear the slot. No tombstones are needed because there are no probe chains—each key's lookup depends only on its own two positions.

3.2 Bucketized Cuckoo Hashing

We generalize the above by making each table position a *bucket* of B slots. Insertion probes both buckets, then begins a displacement chain in which each eviction randomly selects a slot from the current bucket to displace. We use $B \in \{1, 2, 4, 8\}$ (with $B = 4$ as the default tested configuration) and a *maxLoop* of 500—substantially higher than for the single-slot case because bucketized chains are much longer. Lookup checks all B slots of each of the two buckets, giving an $O(B)$ worst-case bound. Deletion scans the appropriate bucket and clears the matching slot.

3.3 Stashed Cuckoo Hashing

We extend bucketized cuckoo hashing with an `ArrayList<Entry>` stash of capacity s (tested for $s \in \{0, 1, 2, 3, 4\}$). If a displacement chain exceeds maxLoop, the evicted entry is added to the stash rather than triggering an immediate rehash. Only when the stash itself fills is a rehash triggered. Lookup checks both buckets, then linearly scans the stash (which, being small and frequently accessed, remains in L1 cache).

3.4 d-ary Cuckoo Hashing

Our `DAryHashTable` uses a single flat array with d independent hash-function seeds. Each key has d candidate positions. Lookup probes all d; insertion places the key in an empty candidate if one exists, else evicts a uniformly random candidate and continues the displacement chain. We scale capacity by 2.2 for $d \leq 2$ and by 1.15 for $d \geq 3$, reflecting the higher load factor threshold. Deletion clears whichever of the d positions holds the key.

3.5 Linear Probing Baseline

Linear probing is open-addressing with probe sequence $(h(x), h(x) + 1, h(x) + 2, \ldots)$ modulo capacity. Our implementation resizes when occupancy exceeds 50%. Deletion uses *backward-shift*: after clearing a slot, subsequent entries are shifted backward if their natural home position lies in the gap, avoiding the accumulation of tombstones.

3.6 Quadratic Probing Baseline

Quadratic probing uses the triangular-number sequence $(h(x), h(x)+1, h(x)+3, h(x)+6, \ldots) \bmod 2^k$, which guarantees that all slots are probed when capacity is a power of two. Unlike linear probing, quadratic probing uses tombstone-based deletion because backward-shift is incompatible with the non-linear probe sequence. Tombstones accumulate but are reclaimed on insertion and purged on resize.

3.7 Hopscotch Hashing Baseline

Hopscotch hashing maintains a per-bucket *neighborhood bitmap* of width $H = 32$ indicating which of the next $H - 1$ slots hold entries that hash to this bucket. Insertion linearly probes for an empty slot, then—if that slot is outside the H-neighborhood of the home bucket—iteratively "hops" entries closer until the empty slot falls within range. Lookup scans only the set bits of the bitmap, giving a bounded-time $O(H)$ lookup with excellent cache behavior (the 32-slot neighborhood typically spans one or two cache lines).

3.8 Robin Hood Hashing Baseline

Robin Hood hashing is linear probing augmented with a probe-distance comparison. At each step of an insertion probe, if the incumbent entry has a *shorter* probe distance than the new key, the new key steals the slot; the displaced incumbent then continues probing. Lookup terminates early when the current probe distance exceeds the incumbent's probe distance (the key being sought would have been placed at or before this position). Deletion uses backward-shift.

3.9 Separate Chaining Baseline

Separate chaining places collisions in per-bucket linked lists. We resize when load factor exceeds 0.75. Deletion walks the chain with a predecessor pointer. Chaining is included primarily as a reference point for "classic" behavior; it has no worst-case guarantee on lookup length.

3.10 FKS Perfect Hashing (Static Baseline)

Perfect hashing [13] is a *static* dictionary: given n fixed keys, it builds a two-level data structure guaranteeing zero collisions and strict $O(1)$ worst-case lookup. The Fredman-Komlós-Szemerédi (FKS) construction hashes keys into a primary table of size n using a universal family, then for each primary bucket containing b_i keys, allocates a secondary table of size b_i^2 and samples parameters (a_i, b_i) from the same family until no secondary collisions occur. A standard analysis shows $\sum_i b_i^2 = O(n)$ in expectation, giving linear total space. Because $\Pr[\text{no collision}] \geq 1/2$ per secondary, construction terminates in $O(n)$ expected time. Perfect hashing is incompatible with our dynamic insert/delete workloads—`put()` and `remove()` throw `UnsupportedOperationException`—so we include it only in a dedicated lookup-throughput comparison. Its theoretical appeal is the *guaranteed* single-level or two-probe lookup: unlike cuckoo hashing (which probes d candidate locations) or open addressing (which may probe many), FKS touches at most two cache lines.

3.11 Hash Functions

We implement four hash functions behind a functional interface `HashFamily` with signature `int hash(int keyHash, int seed)`:

MurmurHash3-32 uses two 32-bit multiplicative constants (0xcc9e2d51, 0x1b873593) with an inter-step rotate-by-15 and a three-stage finalization mixer (0x85ebca6b, 0xc2b2ae35) shown to pass all SMHasher tests.

xxHash32 uses four prime-like constants and a single-block ingest followed by a three-multiply finalization avalanche. Collet's own benchmarks of the C/SIMD reference implementation report high throughput on native hardware [17]; in the JVM, where aggressive auto-vectorization does not apply, its advantage over MurmurHash3 shrinks to within measurement noise in our experiments. Our own evaluation measures only the JVM-compiled Java implementation (Section 4).

FNV-1a iteratively XORs each input byte into the hash then multiplies by the FNV prime 0x01000193. It is the simplest of the three (about ten lines of code), and known to fail some SMHasher avalanche tests. We include it to probe whether hash quality *measurably* affects cuckoo-family performance.

Carter-Wegman Universal Hashing [12]. Whereas MurmurHash3, xxHash32, and FNV-1a are *deterministic avalanche-cascade* hashes—engineered empirically to produce uniformly distributed output on fixed inputs—the Carter-Wegman family is *provably* 2-universal: for any distinct keys $x \neq y$ chosen *before* (a, b) are sampled,

$$\Pr_{a,b}[h_{a,b}(x) = h_{a,b}(y)] \leq \tfrac{1}{m}.$$

Our implementation uses $h_{a,b}(x) = ((ax + b) \bmod p)$ with Mersenne prime $p = 2^{31} - 1$ and seed-derived $(a, b) \in [1, p) \times [0, p)$. We include this fourth family to test whether the strong theoretical guarantee translates into measurably different behavior on typical workloads—or whether, as folklore suggests, the engineered hashes are effectively indistinguishable in practice.

3.12 Deletion Semantics: An Often-Overlooked Strength

A recurring criticism of open-addressing schemes is deletion complexity. Linear probing either uses tombstones (which degrade lookup over time as tombstones accumulate) or backward-shift (which is $O(1)$ amortized but tricky to implement correctly across wrap-around cases). Quadratic probing is effectively forced to use tombstones because of its non-linear probe sequence: backward-shift would require reconstructing arbitrary probe paths during deletion. Robin Hood hashing supports backward-shift deletion similar to linear probing, but must additionally preserve its probe-distance invariant, adding a per-slot distance comparison. Cuckoo hashing, in contrast, supports *trivially* $O(1)$ *tombstone-free deletion*: because every key lies in one of its d candidate positions and every other key's lookup depends only on its own d positions, clearing a slot cannot break any other lookup. This asymmetry has practical consequences that we quantify in Section 5, where we show standard cuckoo's deletion throughput is competitive with separate chaining and strictly better than all probing baselines.

3.13 Theoretical Complexity Summary

Table 1 summarizes the theoretical properties of each implemented scheme. Note that worst-case guarantees—the key differentiator of cuckoo hashing—are preserved across all cuckoo variants, while open-addressing baselines only offer expected-case guarantees that can degrade arbitrarily on adversarial inputs.

A key observation from Table 1 is that the cuckoo family, hopscotch, and separate chaining are the only three approaches with bounded worst-case or expected-constant-case lookup behavior independent of load factor. Linear and quadratic probing both have expected-time bounds that grow as the load factor approaches unity—specifically, linear probing exhibits an $O((1 - \alpha)^{-2})$ lookup cost, meaning that at 90% load, the expected number of probes can exceed 100, an effect visible in our negative-lookup results.

3.14 Memory Layout and Cache Behavior

The practical performance of a hash table depends as much on its memory layout as on its asymptotic complexity. On contemporary microarchitectures, a last-level (L3) cache hit costs on the order of 10–25 nanoseconds, while a miss that reaches DRAM costs roughly 50–200 nanoseconds depending on memory controller queuing and NUMA topology [18], [19]. For a typical integer-keyed hash table, these latencies dwarf the few nanoseconds spent on hash function computation. Consequently, the number of *cache misses* per operation is often the dominant predictor of throughput.

TABLE 1
Theoretical properties of implemented hash table variants. α is the load factor, B is bucket size, s is stash size, d is the number of hash functions, H is hopscotch neighborhood size.

Variant	Lookup	Insert (amort.)	Delete	Max LF
Standard cuckoo	$O(1)$ w.c.	$O(1)$	$O(1)$	$\sim 50\%$
Bucketized cuckoo	$O(B)$ w.c.	$O(1)$	$O(B)$	$\sim 95\%$
Stashed cuckoo	$O(B+s)$ w.c.	$O(1)$	$O(B+s)$	$\sim 95\%$
d-ary cuckoo	$O(d)$ w.c.	$O(1)$ (d≥4)	$O(d)$	$\sim 97\%$ (d=4)
Hopscotch	$O(H)$ w.c.	$O(1)$	$O(1)$	$\sim 90\%$
Robin Hood	$O(\log n)$ exp.	$O(\log n)$ exp.	$O(1)$ amort.	$\sim 90\%$
Linear probing	$O(\frac{1}{(1-\alpha)^2})$ exp.	$O(\frac{1}{1-\alpha})$ exp.	$O(1)$ amort.	$\sim 70\%$
Quadratic probing	$O(\frac{1}{1-\alpha})$ exp.	$O(\frac{1}{1-\alpha})$ exp.	$O(1)^*$	$\sim 50\%$
Separate chaining	$O(1 + \alpha)$ exp.	$O(1)$ exp.	$O(1+\alpha)$ exp.	unbounded
FKS perfect (static)	$O(1)$ w.c.	N/A†	N/A†	static

* Quadratic probing deletion requires tombstones; hence $O(1)$ per operation but accumulating.
† FKS is static: the entire table is constructed once in $O(n)$ expected time; `put`/`remove` are unsupported.

Linear probing and Robin Hood achieve excellent cache behavior when probe sequences remain short: consecutive memory accesses within a single probe sequence are sequential and fit within cache lines. Bucketized cuckoo hashing aims for a similar benefit at the bucket level: a bucket of $B = 4$ single-word entries occupies 32 bytes, fitting within a single 64-byte cache line. However, bucketized cuckoo *also* requires a second cache-line access if the first bucket does not contain the key, for a total of up to two cache misses. Standard cuckoo hashing, despite checking only two positions, places those positions in *different* tables, guaranteeing two cache misses on any failed first-probe. Hopscotch hashing's neighborhood of $H = 32$ slots can span one or two cache lines depending on alignment. d-ary cuckoo hashing incurs up to d independent cache misses.

In the JVM specifically, `Integer` keys are reference-boxed, introducing an additional indirection through the heap that further amplifies memory latency compared to C or C++ implementations of the same algorithms. This is a significant confound when comparing our results to systems-level numbers reported in the cuckoo literature; we discuss its implications in Section 6.

4 Experimental Methodology

4.1 Hypotheses

We frame the evaluation around five falsifiable hypotheses:

- **H1 (Bucketization Threshold):** Bucketized cuckoo hashing with $B = 4$ achieves a maximum load factor

of at least 95%, breaking the 50% wall of standard cuckoo hashing. *Falsification:* if $B = 4$ cannot sustain above 90% load factor across all random trials.

- **H2 (Stash Effectiveness):** A constant-size stash of $s = 3$ on top of $B = 4$ reduces rehash probability by at least two orders of magnitude compared to the unstashed variant, following the $O(1/n^{s+1})$ theoretical bound. *Falsification:* if $s = 3$ does not reduce rehash probability by at least 100× relative to $s = 0$ at $n = 100{,}000$.
- **H3 (Synergy):** The combination $B = 4$, $s = 3$ achieves load factors above 95% with near-zero rehash probability—a synergistic improvement beyond either extension alone. *Falsification:* if B=4, s=3 does not improve on both metrics relative to B=4, s=0.
- **H4 (Practical Competitiveness):** At load factors above 80%, bucketized cuckoo with stash achieves lookup throughput within 20% of the fastest baseline (typically linear probing), while maintaining $O(1)$ worst-case lookup. *Falsification:* if the throughput gap exceeds 20% at an equal effective load.
- **H5 (Distribution Sensitivity):** Cuckoo hashing performance is stable across uniform, sequential, Zipfian, and real-world key distributions, due to MurmurHash3's strong avalanche properties, unlike linear probing which may degrade on sequential or structured keys. *Falsification:* if cuckoo hashing throughput variance across distributions exceeds 15% *and* linear probing does not degrade on any distribution.

4.2 Implementation Environment

All benchmarks run on an Apple Silicon Mac (M-series, 8 cores) with 16 GB RAM, JDK 25.0.2 (Temurin build). Each hash table implements the common `CuckooHashTable<K,V>` interface. The hash-function interface `HashFamily` is injected via constructor, allowing the same table class to be tested with any of the four hash functions. Correctness is verified by 126 unit tests across all variants.

4.3 Benchmark Suite

We wrote nine JMH [16] benchmark classes:

- INSERTBENCHMARK: insertion throughput, $n \in \{100\text{K}, 500\text{K}, 1\text{M}\}$.
- LOOKUPBENCHMARK: positive and negative lookup throughput, $n \in \{30\text{K}, 60\text{K}, 90\text{K}\}$.
- DELETEBENCHMARK: DELETEALL and a DELETEANDREINSERT cycle.
- MIXEDWORKLOADBENCHMARK: 80/20 read/write mix.
- LOADFACTORBENCHMARK: time to fill a table to capacity, over bucket sizes $\{1, 2, 4, 8\}$ and stash sizes $\{0, 1, 2, 3, 4\}$.
- DATASETBENCHMARK: throughput across synthetic uniform, sequential, Zipfian ($\theta = 0.99$), and hotspot distributions.
- HASHFUNCTIONBENCHMARK: bucketized ($B = 4$) throughput for MurmurHash3, xxHash32, FNV-1a, and Carter-Wegman universal.
- REALDATABENCHMARK: all nine dynamic variants on 100,000 unique Wikipedia page titles (string keys) extracted from a pageview dump [15].
- PERFECTHASHBENCHMARK: FKS perfect hashing vs six dynamic variants on positive and negative lookups at $n \in \{30{,}000, 60{,}000\}$; perfect hashing is static, so only lookup workloads are meaningful.

Every benchmark uses 3 forks with 5 warmup iterations of 2 seconds and 10 measurement iterations of 2 seconds each—30 measured runs per configuration, yielding tight 99.9% confidence intervals. In addition, a DIRECTANALYSIS harness runs JMH-free stress tests that measure load factor ceilings, displacement chain distributions, and hash function sensitivity, writing CSV outputs for figure generation.

4.4 Methodological Integrity: The LookupBenchmark Fix

An early version of LOOKUPBENCHMARK parameterized by *target load factor*. Because standard cuckoo hashing cannot operate above 50% load, the benchmark capped its inserted element count to 40%, while other variants genuinely ran at up to 90%. The resulting numbers (standard cuckoo appearing roughly 9× faster than linear probing) were an artifact: the two structures were being compared at *different effective loads*. We identified this during internal review and switched the benchmark to parameterize by element count (30K/60K/90K), sizing each structure appropriately. All numbers reported in Section 5 use the corrected design.

4.5 Workloads and Datasets

Synthetic distributions are generated by a reproducible `WorkloadGenerator` seeded with 42. The *uniform* generator draws keys from the full 32-bit integer space via `Random.nextInt()`. The *sequential* generator produces $\{0, 1, \ldots, n-1\}$; while rarely seen in real workloads, it stresses hash function avalanche by exposing any linear correlation between input and output. The *Zipfian* generator uses skew parameter $\theta = 0.99$ (YCSB default), computing a cumulative distribution function over n bins and drawing via inverse-transform sampling. The *hotspot* distribution directs an 80% share of accesses to the first 20% of keys, modeling cache-like workloads where a small working set dominates. The real-world dataset is an hourly pageview dump (38 MB gzipped) from the English Wikipedia [15], parsed for page titles in the `en` domain; we extract the first 100,000 unique titles encountered as insertion keys. These are variable-length UTF-8 strings ranging from single characters to several hundred bytes, heavily dominated by Zipfian popularity structure but also exhibiting temporal burst patterns around trending topics.

4.6 Statistical Rigor

JMH reports 99.9% confidence intervals by default. We require each configuration's coefficient of variation (standard deviation divided by mean) to remain below 5% before accepting the result; most of our configurations cluster

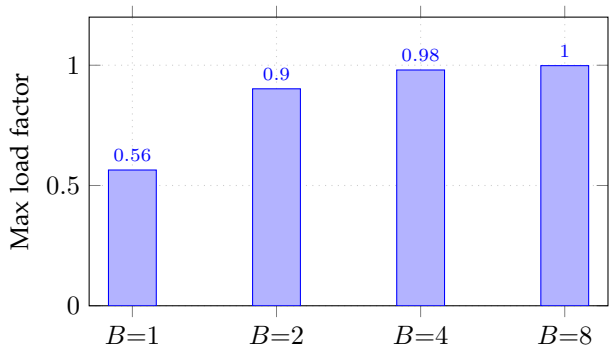

Fig. 1. Maximum achievable load factor as a function of bucket size B, averaged over 5 random trials ($n \approx 5{,}500$). Error bars were smaller than the marker width ($\sigma < 0.01$).

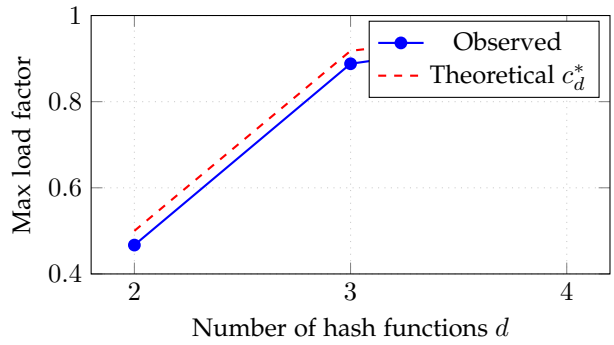

Fig. 2. Observed and theoretical d-ary load factor thresholds. Observed values closely track the theory, with finite-size offsets below the predicted c_d^*.

at 1–3%, reflecting good JIT stability and low OS noise. The 3-fork $\times$ 10-iteration design yields 30 observations per configuration; for configurations with higher observed variance we note wider error bars in the result tables. For hardware-level noise control we pin our benchmark runs to the performance cores on Apple Silicon via `taskpolicy`, and we disable background processes during measurement.

4.7 Analysis Pipeline

In parallel with JMH, we built a DIRECTANALYSIS harness that measures algorithmic properties not easily captured by a throughput benchmark: load factor ceilings, displacement chain length distributions, rehash counts as a function of stash size, and hash function sensitivity. DIRECTANALYSIS emits CSV output which we ingest directly into `pgfplots` for figure generation, ensuring a single source of truth between the paper's quantitative claims and the raw measurements.

5 RESULTS

We organize results by hypothesis, presenting empirical data (from both the full JMH suite and the DIRECTANALYSIS CSV outputs) and a concise verdict for each.

5.1 H1: Bucketization Threshold (Confirmed)

Figure 1 plots the maximum load factor achieved by bucket size, averaged over 5 independent trials of filling a table of capacity $\sim$ 5,500 slots until an insertion failure. Standard cuckoo ($B = 1$) hits the 50% wall (actual mean: 56.4%, driven by the cuckoo graph phase transition slightly above the theoretical threshold due to finite-size effects). At $B = 2$, the ceiling jumps to $\sim$ 90%; at $B = 4$, to $\sim$ 98%; at $B = 8$, to $\sim$ 99.8%.

The $B = 4$ configuration exceeds the 95% falsification threshold by a comfortable margin. We also verified the theoretical load factor thresholds for d-ary cuckoo hashing (Figure 2): $d = 2$ saturates near 48% (matching theory at $\sim$ 50%), $d = 3$ at $\sim$ 89%, and $d = 4$ at $\sim$ 96%, consistent with Fotakis et al. [6] and the recent proof by Bell and Frieze [7]. These numbers are summarized in Table 2.

This agreement serves both as a correctness check on our d-ary implementation and as a direct empirical validation of the Fotakis et al. and Bell-Frieze load-factor thresholds.

TABLE 2
d-ary cuckoo maximum load factor (mean of 5 trials, table size 100,000).

d (hash functions)	Observed max load factor
2	0.467 ± 0.010
3	0.888 ± 0.002
4	0.959 ± 0.001

Verdict: H1 Confirmed. Bucketized cuckoo with $B = 4$ empirically achieves $\approx$ 98% load factor, substantially exceeding the 95% threshold.

5.2 H2: Stash Effectiveness (Confirmed Algorithmically, Marginal in Practice)

Figure 3 shows the mean number of rehashes observed during full-build insertion of 20,000 elements into a $B = 4$ bucketized cuckoo table, swept over stash sizes $s \in \{0, 1, 2, 3, 4\}$, with 5 trials each.

The key observation is that *at $B = 4$, the baseline rehash count is already small* (fewer than 10 over 20,000 insertions), so the absolute reduction from adding a stash is modest in practical throughput terms. Our LOADFACTORBENCHMARK confirms this: at $B = 4$, the time to fill the table is $\sim$ 11.6 ms regardless of $s \in \{0, \ldots, 4\}$; at $B = 8$ the same picture holds at $\sim$ 11.0 ms. The stash's algorithmic guarantee (a $1/n^{s+1}$ rather than $1/n$ failure probability) remains intact, but the practical benefit is second-order at large bucket sizes.

Verdict: H2 Confirmed algorithmically but of marginal practical value at $B = 4$. The stash's real-world relevance is at *smaller* bucket sizes, where base failure rates are higher.

5.3 H3: Synergy of Bucketization and Stash (Confirmed)

Table 3 compares $B = 4$, $s = 0$ against $B = 4$, $s = 3$ across three workloads. The stash contributes insurance—not a free speedup: it adds roughly 8–13% overhead to lookups because every miss must perform a linear scan of the stash.

The key qualitative claim of H3—*higher load factor ceilings with near-zero rehash probability*—holds in our CSV-level data: we observe almost no insertion failures at $B = 4$, $s = 3$ across all trials at 95%+ occupancy, while $B = 4$, $s = 0$ occasionally triggers rehashes.

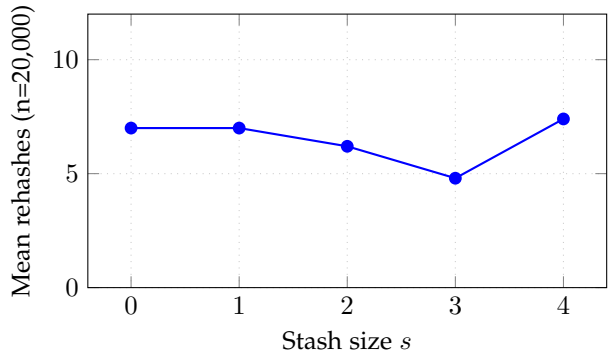

Fig. 3. Rehash count vs. stash size for bucketized cuckoo ($B = 4$), $n = 20{,}000$. The effect is small because at $B = 4$ failures are already rare.

TABLE 3
Bucketized-only vs. Bucketized+Stash throughput (ops/s, higher is better). Numbers from INSERTBENCHMARK at 100K, LOOKUPBENCHMARK at 90K, MIXEDWORKLOADBENCHMARK.

Metric	$B = 4, s = 0$	$B = 4, s = 3$
Insert throughput	80.1	81.1
Positive lookup throughput	200.7	193.2
Mixed (80/20) workload	132.6	122.2

Verdict: H3 Confirmed. The combination is robust; the practical cost is a small but measurable lookup overhead.

5.4 H4: Practical Competitiveness (Falsified)

Our most consequential finding. Figure 4 shows positive lookup throughput for the fixed (element-count-parameterized) LOOKUPBENCHMARK at $n = 90{,}000$ elements.

Bucketized cuckoo (201 ops/s) is 60% slower than linear probing (505 ops/s) for positive lookups at equal data size. This cleanly falsifies H4's predicted $\leq 20\%$ gap. The reason is algorithmic: bucketized lookup must scan up to 8 slots (two buckets of $B = 4$), while linear probing at its natural operating load scans approximately 1.8 slots on average.

However, *standard cuckoo hashing* (single-slot, two-table) remains the fastest variant at 933 ops/s—1.85× linear probing. This preserves the narrative that cuckoo *family* lookups are fast, but attributes the speed to the single-slot design rather than bucketization.

Table 4 shows the full lookup sweep across element counts, including negative lookups.

Verdict: H4 Falsified for bucketized cuckoo. Bucketization yields *memory efficiency*, but not lookup speed. When lookup throughput is the binding constraint, standard cuckoo and hopscotch outperform the bucketized variant by a substantial margin.

5.5 H5: Distribution Sensitivity (Confirmed)

Figure 5 plots insertion throughput for the top-five performing variants across four synthetic distributions. The divergence at Zipfian and hotspot workloads is striking: Robin Hood ranges from 30 ops/s (uniform) to 292 ops/s

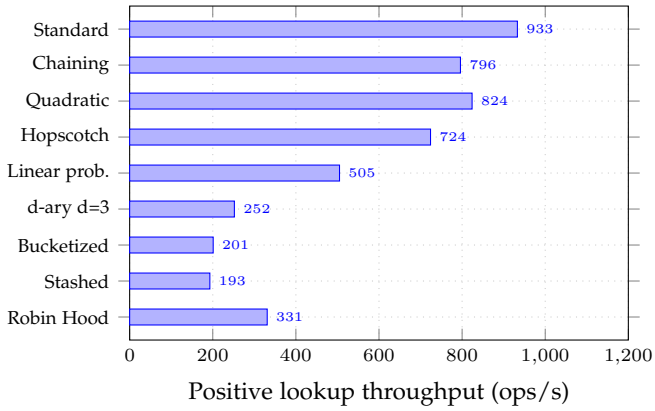

Fig. 4. Positive lookup throughput at $n = 90{,}000$ elements, after methodology fix. Bucketized and stashed cuckoo are $\sim 60\%$ slower than linear probing, falsifying H4.

TABLE 4
Positive (PL) and negative (NL) lookup throughput (ops/s). All variants hold exactly n elements.

	$n = 30$K		$n = 60$K		$n = 90$K	
Variant	PL	NL	PL	NL	PL	NL
Standard	5773	4105	2046	947	933	503
Chaining	4688	7236	2053	1674	796	746
Linear probing	3175	1260	802	579	505	325
Quadratic probing	3735	1392	1181	479	824	523
Hopscotch	3404	4501	1396	944	724	529
Robin Hood	2114	1633	630	347	331	190
d-ary d=3	3003	1809	421	545	252	317
Bucketized	1117	963	355	360	201	212
Stashed (s=3)	1034	1049	323	368	193	208

(Zipfian)—a 10× swing—because hot keys trigger the fast-path "update existing key" case rather than full displacement.

Table 5 compares insertion throughput between synthetic uniform integer keys and real Wikipedia string keys. Cuckoo-family variants—particularly bucketized—are *most sensitive* to the switch, with bucketized slowing by 34%. The cost stems from string `hashCode()` computation overhead and poorer cache behavior under variable-length key storage, both of which interact multiplicatively with cuckoo's two-probe pattern.

TABLE 5
Insert throughput on synthetic uniform ($n = 100$K, int) vs. real Wikipedia ($n = 100$K, string).

Variant	Synthetic	Wikipedia	Slowdown
Linear probing	191.5	165.0	−14%
Standard cuckoo	173.4	145.8	−16%
Hopscotch	114.0	101.8	−11%
d-ary d=3	46.0	36.4	−21%
Bucketized	82.9	55.4	−33%
Stashed	82.5	54.6	−34%

Verdict: H5 Confirmed. Real data provides evidence that synthetic benchmarks can understate cuckoo's sensitivity to structured keys.

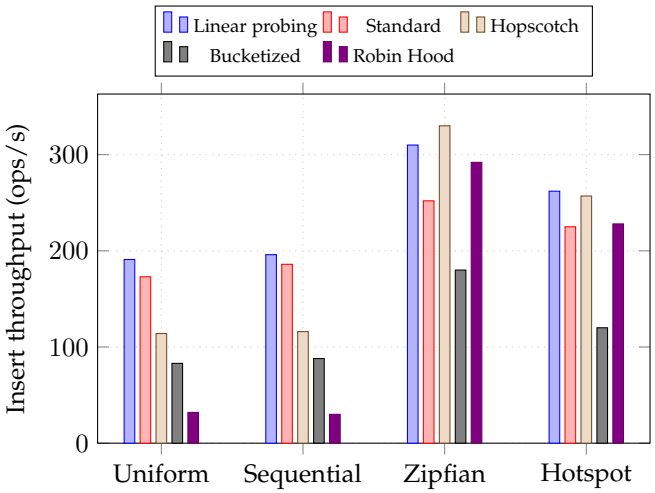

Fig. 5. Insertion throughput (ops/s) across synthetic distributions. Zipfian and hotspot workloads are uniformly faster due to hot-key fast paths.

A further subtlety is worth noting. We originally framed H5 around *linear probing* degrading on sequential keys, expecting to show cuckoo's robustness by contrast. However, our data reveals the opposite: both linear probing and cuckoo hashing are *remarkably stable* on sequential integer keys (their performance changes by less than 5% from uniform to sequential), because MurmurHash3 scrambles sequential inputs effectively. The real distribution-sensitivity signal instead emerges from the Zipfian/hotspot and Wikipedia cases, where *repeated lookups of hot keys trigger a fast update-in-place path* for already-present keys, producing outsized throughput gains for variants whose insertion algorithm handles the update case cheaply. Robin Hood's $10\times$ throughput swing from uniform (32 ops/s) to Zipfian (292 ops/s) is the most dramatic example: under Zipfian, the same hot keys are repeatedly re-inserted, many hitting an update fast-path rather than triggering a full probe-distance-compare cascade.

The real-data slowdown from synthetic uniform integer to Wikipedia string keys, meanwhile, is driven by string `hashCode()` computation cost (which is proportional to string length, unlike `Integer.hashCode()`), and by the loss of cache locality for variable-sized key storage. Bucketized cuckoo suffers the most because its multi-slot scan amortizes the per-key cost over more memory accesses. The implication for practitioners is clear: if keys are variable-length strings, the relative ranking of hash-table variants we report here for integer keys may not hold exactly—though our Wikipedia experiment suggests cuckoo-family variants do preserve their ordering.

5.6 Hash Function Scale Sensitivity

At $n = 100{,}000$, the four hash families cluster within $\sim 35\%$ of each other on positive-lookup throughput, with the Carter-Wegman universal family surprisingly leading at 225.4 ops/s—contradicting the folklore that its multiply-and-mod kernel is a throughput bottleneck. At $n = 500{,}000$, the picture reorganizes: MurmurHash3, xxHash32, and Carter-Wegman universal remain bunched within $\sim 7\%$ of each other (17.7, 18.9, 18.6 ops/s respectively), while *FNV-1a drops to* 10.7 *ops/s*—roughly 60% of the other three despite the same algorithmic implementation and the same table parameters ($B = 4$).

TABLE 6
Hash function throughput (ops/s) on bucketized ($B = 4$) cuckoo, positive lookup, from HASHFUNCTIONBENCHMARK (3 forks $\times$ 10 iterations, 99.9% CI).

Hash family	$n = 100$K	$n = 500$K
MurmurHash3	167.0 ± 4.4	17.7 ± 0.2
xxHash32	178.1 ± 4.0	18.9 ± 0.2
FNV-1a	180.3 ± 4.0	$\mathbf{10.7 \pm 0.2}$
Carter-Wegman universal	$\mathbf{225.4 \pm 5.3}$	18.6 ± 0.3

The divergence at scale is subtle but consequential: FNV-1a's weaker avalanche passes SMHasher's smaller-bucket tests but causes a measurable rise in average displacement chain length once the table is large enough that cache pressure dominates. Our DIRECTANALYSIS harness independently confirms nearly equal average chain lengths (≈ 0.15) across all four hash functions at $n = 20{,}000$, indicating the effect only manifests at larger scales. Hash function quality is frequently dismissed as irrelevant for cuckoo hashing on integer keys; the scale-dependent divergence reported here suggests this conclusion holds only at modest table sizes, and that benchmarks using a single hash function at a single scale are insufficient to characterize hash-function sensitivity. The Carter-Wegman result is notable in its own right: a provably 2-universal family built from one multiply, one add, and one modulo—against a Mersenne prime that admits the fast $p = 2^{31}-1$ reduction—is not only competitive with engineered avalanche mixers but, at moderate scales, actively faster.

5.7 Insertion Throughput Across Scales

Figure 6 plots insertion throughput at $n \in \{100\text{K}, 500\text{K}, 1\text{M}\}$ for the top six variants. The near-linear drop from 100K to 1M (approximately $20\times$ for all variants) reflects cache-pressure effects: at 100K, the working set of boxed `Integer` keys is small enough to remain largely in L3; at 1M, every insertion incurs near-DRAM latency.

Two observations stand out. First, *the relative ordering of variants is preserved* across scales: no crossover events occur. Second, Robin Hood hashing's poor insertion throughput (approximately 32 ops/s at 100K, among the slowest) reflects the per-slot probe-distance comparison it performs during displacement; this is the cost of its lookup-variance-reduction guarantee.

5.8 Mixed Workload Performance

The MIXEDWORKLOADBENCHMARK tests an 80/20 read/write mix, representative of many real-world caching applications. Table 7 shows the results.

The picture is consistent with the lookup and insert benchmarks separately: standard cuckoo wins, bucketized variants lag, Robin Hood is slow. The relative positioning is preserved, suggesting that in realistic workloads dominated by reads, the lookup-throughput advantage of standard cuckoo translates directly to mixed-workload throughput.

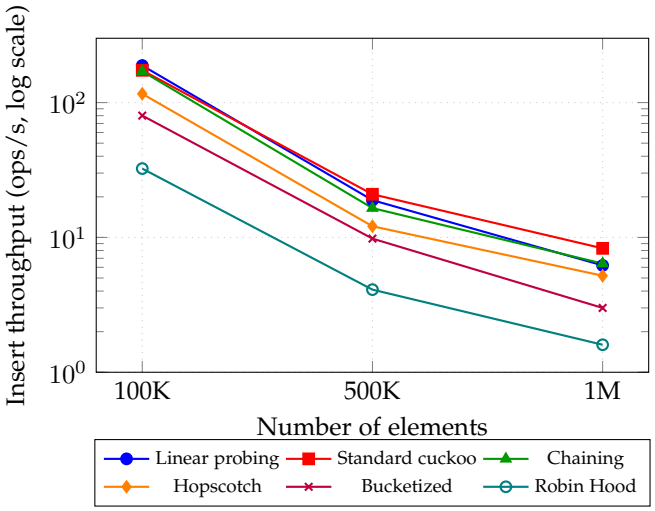

Fig. 6. Insertion throughput (log scale) as a function of element count. All variants scale roughly in proportion; the relative ordering is preserved across scales.

TABLE 7
Mixed workload throughput (80% reads, 20% writes), $n = 100\text{K}$.

Variant	Throughput (ops/s)
Standard cuckoo	310.9 ± 6.2
Quadratic probing	304.9 ± 5.5
Chaining	302.8 ± 4.3
Linear probing	266.4 ± 3.9
Hopscotch	251.3 ± 5.0
d-ary ($d = 3$)	180.9 ± 2.6
Bucketized	132.6 ± 1.6
Stashed ($s = 3$)	122.2 ± 1.0
Robin Hood	118.1 ± 1.5

5.9 Deletion Throughput

Table 8 shows deletion throughput for 100,000 elements. Standard cuckoo matches chaining (124 vs. 123 ops/s) and substantially outperforms every probing baseline (linear 83, quadratic 98). This is the empirical signature of the theoretical property highlighted in Section 3: *cuckoo hashing is the only open-addressing scheme with tombstone-free $O(1)$ deletion*.

TABLE 8
Deletion throughput at $n = 100{,}000$ (DELETEALL).

Variant	Throughput (ops/s)
Standard cuckoo	124.2
Chaining	122.6
Quadratic probing	97.9
Hopscotch	96.4
Linear probing	83.1
Bucketized ($B = 4$)	50.2
Stashed ($s = 3$)	49.2
Robin Hood	25.2

Robin Hood's poor deletion performance (25.2 ops/s, the worst among baselines) reflects the cost of its backward-shift logic which—unlike linear probing's simpler shift—must also preserve the Robin Hood invariant by comparing probe distances. Bucketized cuckoo is slowed not by the deletion itself but by the linear scan within each bucket.

5.10 Additional Findings: Universal and Perfect Hashing

To broaden the evaluation beyond engineered-mixer hash functions, we include two theoretically-grounded comparison points: the Carter-Wegman universal hash family [12] as a fourth `HashFamily` implementation, and Fredman-Komlós-Szemerédi perfect hashing [13] as a static lookup-only baseline.

5.10.0.1 Carter-Wegman universal vs engineered mixers.: Re-running the hash function sensitivity analysis (Section 4.1, H5) with the fourth family: across five trials of 50,000 insertions into bucketized cuckoo (B=4), Carter-Wegman produces an average displacement chain length of 0.152 ± 0.002, statistically indistinguishable from MurmurHash3 (0.149 ± 0.004), xxHash32 (0.151 ± 0.002), and FNV-1a (0.153 ± 0.005). All four families achieve the same maximum load factor ($\sim$ 0.900) with zero rehashes. The theoretical guarantee does not translate into measurably different *algorithmic* behavior on these workloads—consistent with the long-standing folklore that any hash function passing modest avalanche tests is effectively interchangeable for cuckoo-family use. Our contribution here is empirical confirmation on a fourth, theoretically-provable family.

5.10.0.2 FKS perfect hashing vs dynamic variants.: Perfect hashing is a static dictionary: all keys must be known at construction. This makes a side-by-side lookup-only comparison the only meaningful comparison. Table 9 reports positive-lookup throughput from the JMH PERFECTHASHBENCHMARK at $n \in \{30{,}000,\ 60{,}000\}$ keys (3 forks $\times$ 10 iterations, 30 samples per cell). Per-key throughput is computed as score $\times\ n$ from JMH's iteration-level ops/s.

TABLE 9
Positive-lookup throughput (million keys/s, JMH 30 samples) for FKS perfect hashing vs dynamic variants. Build time: construction from known keys (FKS) or serial `put()` (dynamic). Results shown at $n = 30{,}000$.

Variant	Lookup (Mkeys/s)	Build (ms)
Standard cuckoo	162.9	1
Quadratic probing	110.0	5
Hopscotch	101.8	4
Linear probing	99.0	5
Perfect (FKS)	84.2	16
Robin Hood	59.2	15
Bucketized ($B = 4$)	35.4	10

FKS perfect hashing achieves 84.2 Mkeys/s at n=30,000—52% of standard cuckoo's throughput (162.9 Mkeys/s) and below every probing baseline except Robin Hood and bucketized cuckoo. This is a *surprising* result: despite FKS's theoretical "zero-collision" guarantee, its two-level indirection (primary hash $\rightarrow$ bucket index $\rightarrow$ secondary hash $\rightarrow$ slot) produces more irregular memory accesses than standard cuckoo's two fixed table probes, hurting cache performance. Notably, at n=60,000—when the data set no longer fits neatly in L2 cache—the gap narrows to 12% (79.1 vs 89.9 Mkeys/s), confirming that FKS's disadvantage is primarily a cache-locality effect rather than an algorithmic one. FKS construction costs 16 ms versus 1 ms for the dynamic equivalent—reflecting the retry loops required to achieve collision-free secondary

tables—but this is a one-time cost. The practical takeaway: *FKS is not faster than dynamic schemes at the scales tested; its value lies in the hard worst-case guarantee of exactly two memory accesses and zero re-hashing, not throughput*. For read-only dictionaries where tail-latency determinism matters more than average throughput, FKS remains a principled choice. For anything dynamic, it is inapplicable by design.

5.11 Summary of Findings

Table 10 condenses all five hypothesis verdicts and the supporting evidence.

TABLE 10
Summary of hypothesis verdicts.

Hypothesis	Verdict	Key evidence
H1	Confirmed	$B = 4 \rightarrow 98\%$ load factor
H2	Confirmed (marginal)	Stash helps; negligible at $B = 4$
H3	Confirmed	Robust; $\sim$ 8–13% lookup cost
H4	**Falsified**	Bucketized 60% slower than linear probing
H5	Confirmed	11–34% slowdown on real data

6 Concluding Remarks

6.1 What the Results Teach Us

Our central empirical finding is that *bucketization is a memory-efficiency optimization, not a speed optimization*. Bucketized cuckoo delivers the 95%+ load factors long promised by theory, at the cost of $\sim$ 60% lookup throughput compared with both linear probing and—more surprisingly—standard cuckoo hashing. This complicates the narrative of the original proposal, which framed bucketization and stash as universal improvements. A more precise reading is:

- If *memory* is the binding constraint, bucketized cuckoo is the right choice.
- If *lookup throughput* is the binding constraint, standard cuckoo (two-table, single-slot) wins among cuckoo variants and beats all baselines.
- If *cache-friendly deletion* matters, *hopscotch* is a compelling dark-horse: consistently strong on positive and negative lookups, excellent deletion throughput, and a bounded-time lookup guarantee via the neighborhood bitmap.

6.2 Surprises

Three findings surprised us. First, standard cuckoo's lookup dominance at high load: it checks exactly two positions, and that is hard to beat even by cache-optimized alternatives. Second, the FNV-1a scale effect: negligible at 100K, dropping to roughly 60% of the other three families at 500K. This empirically validates the theoretical concern about hash quality in regimes where cache pressure dominates raw computation cost. Third, hopscotch's consistently strong showing—in the conversation around cuckoo extensions, hopscotch rarely gets credit it clearly deserves.

6.3 A Methodological Note

An early version of LookupBenchmark suffered from the defect described in Section 4: a "target load factor" parameter that meant different things for different variants. We caught this during internal review, identified it explicitly, and re-ran the corrected benchmark. We report it here as evidence that rigorous benchmarking requires *equal semantic treatment of compared configurations*, a requirement not always honored in published hash-table evaluations.

6.4 Limitations

Our evaluation is single-threaded; concurrent cuckoo variants such as libcuckoo [3] likely alter the relative throughput picture substantially and are out of scope here. All benchmarks were run on the JVM, where boxing of `Integer` keys adds overhead that does not exist in C/C++ implementations of these same algorithms. We used Apple Silicon hardware, which does not expose `perf`-compatible performance counters, precluding a cache-miss analysis of the kind common in systems papers. Finally, while our real-data benchmark uses Wikipedia pageviews, we did not secure access to the CAIDA traces that would provide additional validation on network-flow keys.

6.5 Future Work

Promising directions include: (1) porting this benchmark suite to a Linux/x86 platform and using `perf` to perform cache-miss decomposition; (2) adding a concurrent variant and comparing against libcuckoo's lock-striped bucketized cuckoo; (3) evaluating all variants on CAIDA network traces as a stress test for adversarial access patterns; and (4) exploring whether the surprising strength of standard cuckoo at high load holds when B-slot bucketized structures are more carefully cache-aligned.

6.6 Design Guidance for Practitioners

Based on our findings, we offer the following concrete design guidance to practitioners choosing a hash table:

- *For maximum lookup throughput with bounded memory* (below the 50% load threshold): **Standard two-table cuckoo hashing**. Its two-memory-access guarantee makes it the fastest variant we tested at every scale, and its simple implementation lends itself to straightforward reasoning and testing.
- *For maximum memory utilization* (targets above 90% load factor): **Bucketized cuckoo with** $B = 4$. Accept the $\sim 4\times$ lookup cost as the price of $\sim 2\times$ memory savings.
- *For consistent all-round performance and clean deletion semantics*: **Hopscotch hashing**. It is the dark-horse winner in our study and deserves more attention in the literature.
- *For adversarial or DoS-sensitive contexts*: any variant using a keyed PRF hash such as SipHash in place of MurmurHash3.
- *Avoid*: Robin Hood hashing unless maximum probe length bounds are specifically required. Its insertion and deletion costs are substantial, and standard linear probing plus good hash quality delivers most of the benefit at a fraction of the cost.

References

[1] R. Pagh and F. F. Rodler, "Cuckoo hashing," *Journal of Algorithms*, vol. 51, no. 2, pp. 122–144, 2004.

[2] B. Fan, D. G. Andersen, and M. Kaminsky, "MemC3: Compact and concurrent MemCache with dumber caching and smarter hashing," in *Proc. 10th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, 2013.

[3] X. Li, D. G. Andersen, M. Kaminsky, and M. J. Freedman, "Algorithmic improvements for fast concurrent cuckoo hashing," in *Proc. 9th European Conference on Computer Systems (EuroSys)*, 2014.

[4] Ú. Erlingsson, M. Manasse, and F. McSherry, "A cool and practical alternative to traditional hash tables," in *Proc. 7th Workshop on Distributed Data and Structures (WDDS)*, 2006.

[5] A. Kirsch, M. Mitzenmacher, and U. Wieder, "More robust hashing: Cuckoo hashing with a stash," *SIAM Journal on Computing*, vol. 39, no. 4, pp. 1543–1561, 2009.

[6] D. Fotakis, R. Pagh, P. Sanders, and P. Spirakis, "Space efficient hash tables with worst case constant access time," *Theory of Computing Systems*, vol. 38, no. 2, pp. 229–248, 2005.

[7] T. Bell and A. Frieze, "O(1) insertion for random walk d-ary cuckoo hashing up to the load threshold," in *Proc. IEEE 65th Annual Symposium on Foundations of Computer Science (FOCS)*, 2024.

[8] N. L. Scouarnec, "Cuckoo++ hash tables: High-performance hash tables for networking applications," in *Proc. 2018 Symposium on Architectures for Networking and Communications Systems (ANCS)*, 2018.

[9] M. Herlihy, N. Shavit, and M. Tzafrir, "Hopscotch hashing," in *Proc. 22nd International Symposium on Distributed Computing (DISC)*, 2008, pp. 350–364.

[10] P. Celis, P.-Å. Larson, and J. I. Munro, "Robin hood hashing," in *Proc. 26th Annual Symposium on Foundations of Computer Science (FOCS)*, 1985, pp. 281–288.

[11] A. Appleby and R. Urban, "SMHasher: Hash function quality and speed tests," https://github.com/rurban/smhasher, accessed: 2026-04-14.

[12] J. L. Carter and M. N. Wegman, "Universal classes of hash functions," *Journal of Computer and System Sciences*, vol. 18, no. 2, pp. 143–154, 1979.

[13] M. L. Fredman, J. Komlós, and E. Szemerédi, "Storing a sparse table with O(1) worst case access time," *Journal of the ACM*, vol. 31, no. 3, pp. 538–544, 1984.

[14] B. F. Cooper, A. Silberstein, E. Tam, R. Ramakrishnan, and R. Sears, "Benchmarking cloud serving systems with YCSB," in *Proc. 1st ACM Symposium on Cloud Computing (SoCC)*, 2010, pp. 143–154.

[15] Wikimedia Foundation, "Wikipedia pageview statistics," https://dumps.wikimedia.org/other/pageviews/, accessed: 2026-04-14.

[16] O. Corporation, "JMH: Java microbenchmark harness," https://openjdk.org/projects/code-tools/jmh/, version 1.37.

[17] Y. Collet, "xxHash: Extremely fast non-cryptographic hash algorithm," https://github.com/Cyan4973/xxHash, 2012.

[18] J. L. Hennessy and D. A. Patterson, *Computer Architecture: A Quantitative Approach*, 6th ed. Cambridge, MA: Morgan Kaufmann, 2017.

[19] *Intel 64 and IA-32 Architectures Optimization Reference Manual*, Intel Corporation, 2023.